Handwritten Digit Recognition Using a Neural Network

1. Problem Being Solved

The objective of this project is to develop a deep learning model that can recognize handwritten digits from 0 to 9. Handwritten digit recognition is useful in applications such as postal code reading, bank cheque processing, and digit recognition in scanned documents. The model analyzes pixel values from an image and predicts the corresponding digit.

2. Dataset Description

The project uses a CSV file named mnist_data.csv, containing 8,035 rows and 785 columns before cleaning. Each image is represented by 784 pixel features, corresponding to a 28 × 28 grayscale image, and one label column representing the actual digit. The dataset initially contained 75 missing values, which were handled during preprocessing. After cleaning, no missing values remained.

```
df.shape
```

```
(8035, 785)
```

```
df.head()
```

	label	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	...	28x19	28x20	28x21	28x22	28x23	28x24	28x25	28x26	28x27	28x28
0	5	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	4	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	1	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	9	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 785 columns

3. Neural Network Architecture

The model was developed using TensorFlow and Keras. It consists of

- an input layer with 784 features,
- a hidden Dense layer containing 128 neurons with ReLU activation,
- a Dropout layer with a rate of 20%,
- a second Dense layer with 64 neurons and ReLU activation,
- an output layer with 10 neurons using Softmax activation.

The output layer predicts the probabilities of the ten digit classes.

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

```
Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)
```

4. Training Process

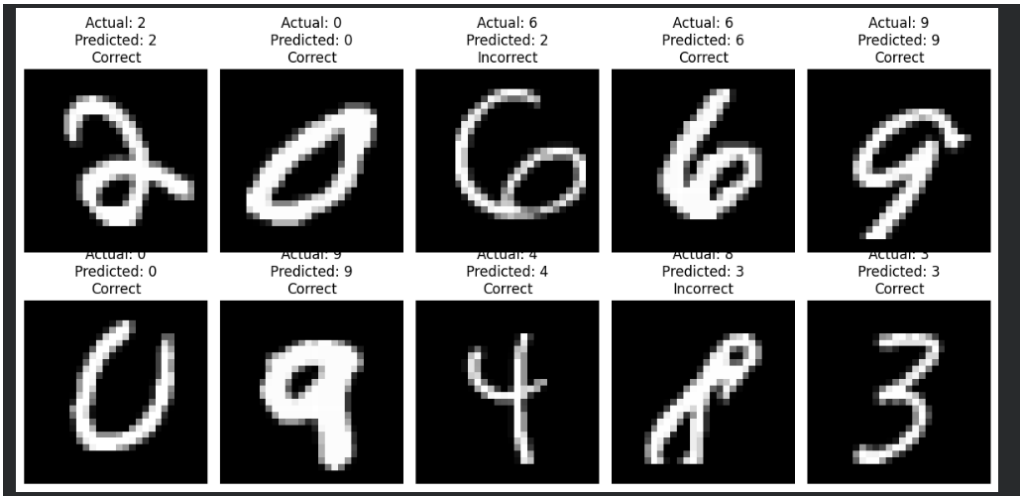
1. The pixel values were normalized by dividing them by 255, bringing them into the range of 0 to 1.
2. The dataset was divided into 80% training data and 20% testing data using a stratified split.
3. The model was compiled with the Adam optimizer, sparse categorical cross-entropy loss, and accuracy as the evaluation metric.
4. Training was performed for 10 epochs with a batch size of 128. Ten percent of the training portion was used for validation.

```python
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)
```

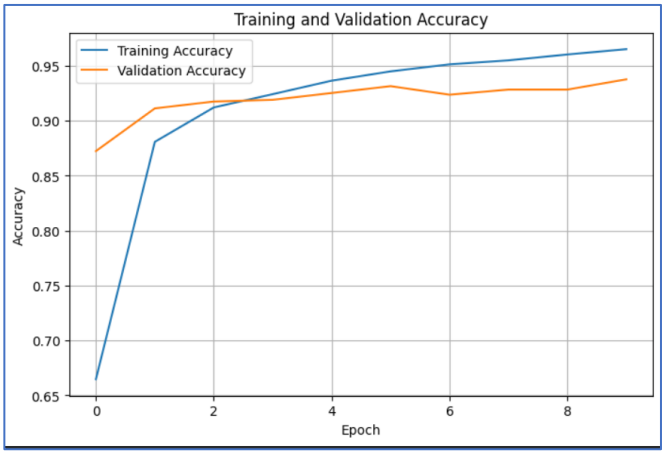
```
Epoch 1/10
46/46 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.6648 - loss: 1.1699 - val_accuracy: 0.8725 - val_loss: 0.4431
Epoch 2/10
46/46 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8809 - loss: 0.4174 - val_accuracy: 0.9114 - val_loss: 0.3417
Epoch 3/10
46/46 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9122 - loss: 0.3093 - val_accuracy: 0.9176 - val_loss: 0.2816
Epoch 4/10
46/46 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9243 - loss: 0.2640 - val_accuracy: 0.9191 - val_loss: 0.2558
Epoch 5/10
46/46 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9366 - loss: 0.2257 - val_accuracy: 0.9253 - val_loss: 0.2395
Epoch 6/10
46/46 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9450 - loss: 0.1909 - val_accuracy: 0.9316 - val_loss: 0.2370
Epoch 7/10
46/46 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9514 - loss: 0.1707 - val_accuracy: 0.9238 - val_loss: 0.2363
Epoch 8/10
46/46 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9551 - loss: 0.1523 - val_accuracy: 0.9285 - val_loss: 0.2194
Epoch 9/10
46/46 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9604 - loss: 0.1337 - val_accuracy: 0.9285 - val_loss: 0.2272
Epoch 10/10
46/46 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9653 - loss: 0.1198 - val_accuracy: 0.9378 - val_loss: 0.1961
```

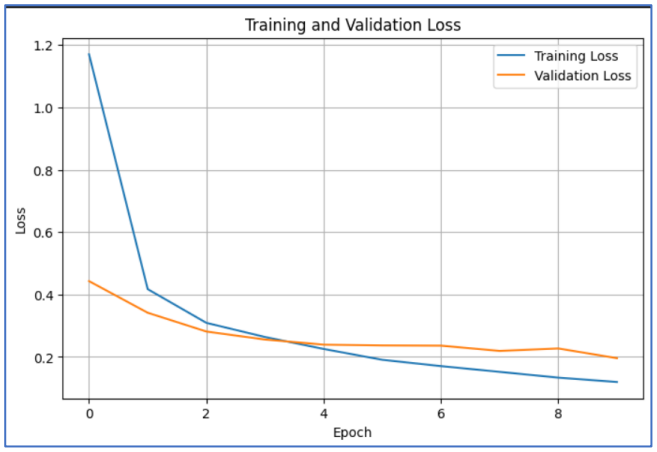
5. Results: The model achieved a test accuracy of **94.34%** and a test loss of **0.193**.

```
Test Accuracy: 94.34 %
Test Loss: 0.193
```

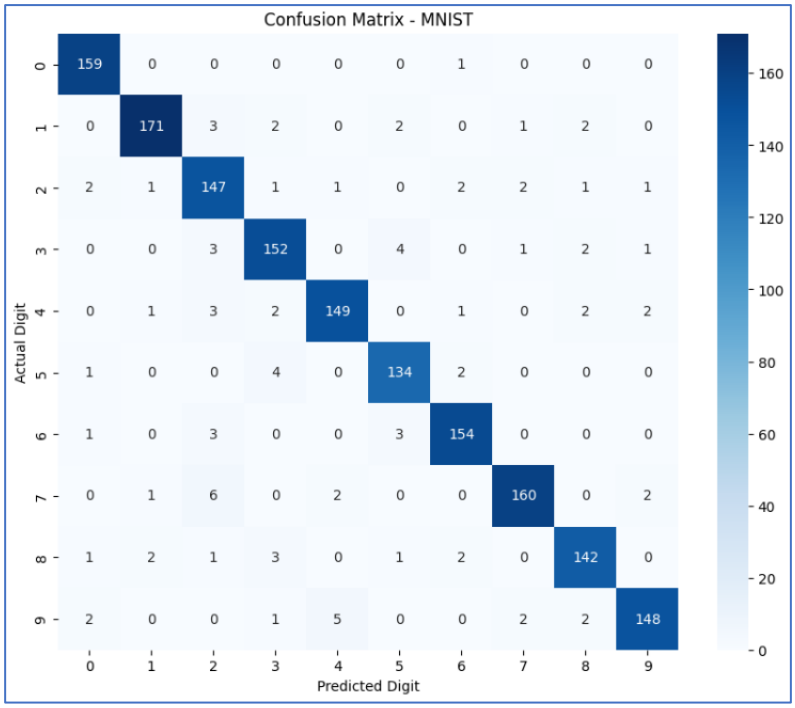


Training and validation accuracy and loss graphs were plotted to monitor learning.





A confusion matrix, classification report, sample predictions, and misclassified images were also generated. Out of 1,607 test examples, 91 were incorrectly classified.



6. Limitations

- The dataset used contains only 8,035 examples, which is smaller than the complete MNIST dataset.
- The model may struggle with unclear handwriting, unusual writing styles, and digits with similar shapes.
- Additionally, the fully connected neural network does not directly exploit the spatial relationships between neighboring pixels.

7. Future Improvements

- Future work could use a Convolutional Neural Network (CNN), which is better suited to image recognition.
- Increasing the training dataset, applying data augmentation, and tuning hyperparameters may improve accuracy and generalization.
- Early stopping and additional evaluation on unseen handwritten images could also help assess the model's reliability.